Data Structures Fall 2023

Lab 12: AVL Trees with Unit Testing

AVL Trees:

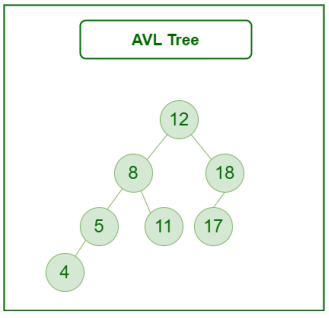
An AVL tree, named after its inventors Adelson-Velsky and Landis, is a self-balancing binary search tree. The key feature of AVL trees is that they maintain balance during insertions and deletions, ensuring that the tree remains relatively balanced, which helps in maintaining efficient search, insert, and delete operations.

In an AVL tree, for every node, the heights of the left and right subtrees differ by at most one. This balance condition is enforced after every insertion and deletion operation through a process called tree rotation. Tree rotations are local operations that maintain the balance of the tree while preserving the order of the elements.

The AVL tree's balancing property ensures that the height of the tree is logarithmic with respect to the number of nodes, resulting in efficient search times. The worst-case time complexity for basic operations (search, insert, delete) in an AVL tree is O(log n), where n is the number of nodes.

In summary, AVL trees provide a balanced and efficient data structure for dynamic sets or ordered maps, offering guaranteed logarithmic height and, consequently, efficient search and update operations.

Representation of AVL Tree:



Task 1:

Define a C++ class for a AVL Tree that includes a Node class **using templates**. The Node class should have data, a pointer to the left child, a pointer to the right child and integer height. Implement the following operations for the AVL Tree **using templates**:

1. **AVLNode:** Implement a structure for a node in an AVL tree. Include fields for data, a pointer to the left child, a pointer to the right child, and the height of the node.

2. **AVL Tree Insertion:**Write a function to insert a new node into an AVL tree. Ensure that the tree remains balanced after the insertion by performing rotations if necessary

3.**AVL Tree Deletion**:Implement a function to delete a node with a given key from an AVL tree. Ensure that the tree remains balanced after the deletion by performing rotations if necessary

4. **AVL Tree Height:**Write a function to calculate the height of a node in an AVL tree.

5. **AVL Tree Balance Factor**:Implement a function to calculate the balance factor of a node in an AVL tree. The balance factor is the height of the left subtree minus the height of the right subtree.

6**. Check if AVL Tree:**Write a function to check if a given binary search tree is also an AVL tree. Ensure that the heights of the left and right subtrees differ by at most 1 for every node.

7. **AVL Tree Traversal:**Implement recursive functions for in-order, pre-order, and post-order traversal of an AVL tree.

8. **Find Smallest and Largest Elements in AVL Tree**:Write functions to find the smallest and largest elements in an AVL tree.

9. **AVL Tree Rotation:**Implement functions for left and right rotations in an AVL tree.

10. **AVL Tree Rebalancing:** Write a function that performs the necessary rotations to rebalance a node in an AVL tree after an insertion or deletion.

Task 2:

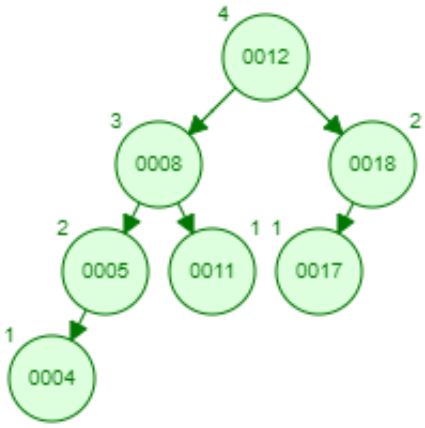
Question: Suppose you have an AVL tree containing distinct integer keys. Design an algorithm to split this AVL tree into two separate AVL trees, TreeA and TreeB, such that TreeA contains keys less than or equal to a given threshold value K, and TreeB contains keys greater than K. The algorithm should ensure that both resulting trees maintain the AVL property.

Note: The new sub trees formed after splitting can have multiple variations, but it should be an AVL tree.

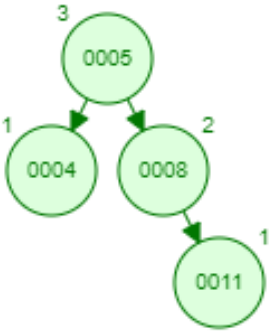
Example:

In this example the threshold value is 0012, so we will be splitting the tree on the value 12, all values in the left sub tree of 12 will make an independent tree & right sub tree will be an independent tree.

Original Tree:



After splitting:
Left tree:



Right Tree:

Note: The order of elements in the left & right tree can differ based upon the on the order of insertion in the tree.